COMPLETE STUDY MANUAL

Retrieval-Augmented Generation (RAG)

+ LangGraph Crash Course

Detailed notes: theory, architecture, evaluation, failure modes

Project 1: complete RAG system from scratch (LangChain + Chroma)

Project 2: agentic RAG with LangGraph (routing, grading, retries, memory)

Full source code - cheat sheets - interview Q&A - exercises

Python 3.10+ | LangChain | LangGraph | Chroma | OpenAI or local (Ollama)

October 2026 edition

Table of Contents

Introduction . . . 3

About this manual . . . 3

Part 1: Retrieval-Augmented Generation (RAG) . . . 5

1. Why RAG? The problem it solves . . . 6
2. RAG architecture end to end . . . 8
3. Building blocks in depth . . . 10
4. LangChain essentials you need for RAG . . . 14
5. Environment setup . . . 15

Part 2: Project 1 RAG from Scratch . . . 16

6. Project 1 overview: a document Q&A; assistant . . . 17
7. Project 1 - source code . . . 18
8. Project 1 - run it and understand it . . . 25

Part 3: LangGraph Crash Course . . . 27

9. Why LangGraph? . . . 28
10. Core concepts . . . 29
11. Your first graph (no LLM needed) . . . 30
12. A chatbot with memory . . . 31
13. Tools and the agent loop . . . 32
14. Streaming, human-in-the-loop and design rules . . . 33
15. From naive RAG to agentic RAG with LangGraph . . . 35

Part 4: Project 2 Agentic RAG with LangGraph . . . 36

16. Project 2 overview: a self-correcting RAG agent . . . 37
17. Project 2 - source code . . . 40
18. Project 2 - run, trace and understand . . . 53

Part 5: Reference and Practice . . . 56

19. Cheat sheets . . . 57
20. Interview questions and model answers . . . 59
21. Practice exercises . . . 61
22. Glossary . . . 62
23. Where to go next . . . 63

About this manual

This manual teaches **Retrieval-Augmented Generation (RAG)** and **LangGraph** from first principles and then makes you build two complete, working projects. Everything is in this one PDF: theory you can revise from, full source code you can copy, and practice material to check your understanding.

Who it is for

Students and developers who know basic Python (functions, classes, virtual environments) and have used an LLM chat product. No prior LangChain or LangGraph knowledge is assumed.

What is inside

Part	Contents	Outcome
1 - RAG theory (Ch. 1-5)	Why RAG, architecture, chunking, embeddings, vector stores, retrieval strategies, prompting, evaluation, failure modes, LangChain essentials, setup	You can explain and design a RAG system
2 - Project 1 (Ch. 6-8)	Complete document Q&A with LangChain + Chroma, retrieval evaluation script	A working RAG app
3 - LangGraph (Ch. 9-15)	State, nodes, edges, reducers, memory, tools, streaming, human-in-the-loop, agentic RAG patterns	You can build stateful graphs
4 - Project 2 (Ch. 16-18)	Agentic RAG: routing, grading, query rewriting, hallucination check, memory, live trace	A self-correcting RAG agent
5 - Reference (Ch. 19-23)	Cheat sheets, 16 interview Q&A, exercises, glossary, next steps	Revision and practice

Suggested 5-day study plan

Day	Do this	Check yourself
1	Read Chapters 1-5. Set up the environment.	Can you draw the two RAG phases from memory?
2	Build Project 1 (Ch. 6-8). Run the experiments in 8.4.	What happens to hit rate if chunk size doubles?
3	Read Chapters 9-15. Run the hello graph, chatbot and tool agent.	What is a reducer? When does a conditional edge run?
4	Build Project 2 (Ch. 16-18). Trace all six test paths.	Why is `finalize` the single exit node?
5	Evaluation, exercises (Ch. 21), interview questions (Ch. 20).	Explain Corrective, Self and Adaptive RAG in 1 minute each.

Conventions

- `monospace` text is code, file names or commands. Boxed listings with a file name caption are complete files.
- Colored boxes: **KEY IDEA** (must remember), **TIP** (helpful shortcut), **WARNING / PITFALL** (common mistake), **NOTE**.
- Console output marked *illustrative* shows the shape of results; exact wording depends on the model you use.

- Both projects run on OpenAI **or** fully local models (Ollama) by changing one line in `.env`.

READ ME

Honest note on testing: the source files were syntax-checked, and the data flow was reviewed line by line, but LangChain and LangGraph release often. If a package upgrade renames an import, consult the official release notes or pin the versions you installed.

PART 1

Retrieval-Augmented Generation (RAG)

Theory, architecture, building blocks, evaluation and failure modes

1. Why RAG? The problem it solves

Large Language Models (LLMs) are impressive, but when you put one into a real product you quickly meet five hard limits. RAG is the standard engineering answer to all five.

LLM limitation	What it looks like in practice	How RAG helps
Knowledge cut-off	Model does not know last month's policy change or this week's price list.	Facts come from your index, which you can update any time.
No private data	It has never seen your wiki, contracts, tickets or PDFs.	Your documents are indexed and retrieved at query time.
Hallucination	Fluent but invented answers, fake numbers, fake citations.	The prompt forces the model to answer from retrieved text and say 'I don't know'.
No traceability	You cannot tell where a claim came from.	Each answer can cite the exact chunk, file and page it used.
Cost of long context	Pasting 500 pages in every prompt is slow and expensive.	Only the 3-5 most relevant chunks are sent.

1.1 Definition

> **DEFINITION**
> **Retrieval-Augmented Generation** is a pattern in which, at question time, the system first *retrieves* relevant passages from an external knowledge source and then *augments* the LLM prompt with those passages so that the model *generates* an answer grounded in them.

The term comes from the 2020 paper *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (Lewis et al., Facebook AI Research). Today it usually means the simple, practical form you will build in this manual: embed documents, store vectors, retrieve by similarity, and prompt a chat model.

Mental model - the open-book exam. A plain LLM takes a closed-book exam from memory. A RAG system takes an open-book exam: before answering, it looks up the relevant pages in the textbook (your documents) and writes the answer using them.

1.2 RAG vs fine-tuning vs long-context prompting

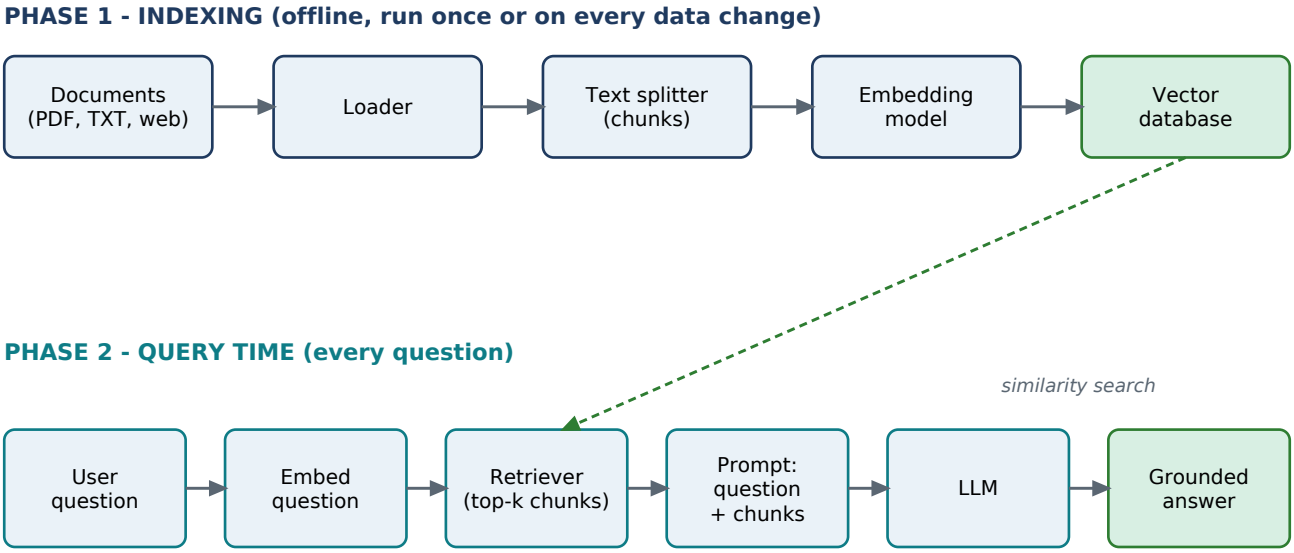
Aspect	RAG	Fine-tuning	Long-context prompting
Best for	Facts, documents, changing knowledge	Style, format, domain skills, behaviour	One-off analysis of a few documents
Freshness	Instant: re-index a file	Slow: retrain to update	Instant, but must resend every time
Cost per query	Low (few chunks)	Low at inference, high to train	High (all tokens every call)
Hallucination control	Good (grounded + citations)	Weak for facts	Medium
Citations	Natural	Not available	Possible but manual
Data access control	Easy (filter per user)	Hard (knowledge baked into weights)	Per request
Typical effort	Days	Weeks	Minutes

TIP
RAG and fine-tuning are not rivals. Use RAG for *what the model should know* and fine-tuning for *how the model should behave*. Many production systems use both.

1.3 When RAG is the wrong tool

- The answer requires **reasoning over the whole corpus** (e.g. 'summarise all 2,000 contracts'). Plain top-k retrieval only sees a few chunks; you need map-reduce summarisation or a structured database.
- The data is **highly structured** (orders, metrics). Use SQL / an analytics tool and let the LLM generate the query instead of embedding rows of numbers.
- The knowledge is **small and stable** (a 2-page FAQ). Just put it in the system prompt.

2. RAG architecture end to end



The same embedding model must be used in both phases.

2.1 Phase 1 - Indexing (offline)

1. **Load** raw sources (PDF, Word, HTML, Markdown, databases, APIs) into `Document` objects: `page_content` (text) + `metadata` (source, page, author, date...).
2. **Split** each document into chunks small enough to embed and to fit in the prompt, with some overlap so that sentences cut at a boundary are still found.
3. **Embed** every chunk with an embedding model: text in, a vector of floating-point numbers out. Texts with similar meaning get nearby vectors.
4. **Store** vectors + text + metadata in a vector database with an approximate-nearest-neighbour index.

2.2 Phase 2 - Retrieval and generation (online)

1. **Embed the question** with the *same* embedding model.
2. **Retrieve** the top-k most similar chunks (optionally filtered by metadata, re-ranked, or merged with keyword search).
3. **Augment**: build a prompt that contains instructions, the retrieved chunks (numbered) and the question.
4. **Generate**: call the LLM at low temperature; return the answer together with source citations.

2.3 Three generations of RAG

Level	Flow	Strengths	Weaknesses
Naive RAG	question -> retrieve -> generate (a fixed pipeline)	Simple, fast, cheap. Project 1.	Retrieves even when unnecessary; cannot recover from bad retrieval.

Level	Flow	Strengths	Weaknesses
Advanced RAG	adds query rewriting, hybrid search, re-ranking, compression, metadata filters	Noticeably better recall and precision.	More components to tune and monitor.
Agentic RAG	an LLM-driven *graph*: route, grade, rewrite, retry, self-check, use tools, remember	Self-correcting, handles multi-step questions. Project 2 (LangGraph).	More LLM calls: higher latency and cost; needs loop guards.

3. Building blocks in depth

3.1 Document loading

Loaders convert files into `Document` objects. LangChain ships dozens (`PyPDFLoader`, `TextLoader`, `WebBaseLoader`, `CSVLoader`, `UnstructuredWordDocumentLoader`, Notion, Google Drive, S3 ...). Two rules matter:

- **Garbage in, garbage out.** Check the extracted text. Scanned PDFs need OCR; multi-column PDFs and tables often come out scrambled. Clean headers, footers and page numbers.
- **Keep metadata.** `source`, `page`, `title`, `section`, `date`, `access_level` enable citations, filtering and debugging later. Metadata is cheap now and impossible to reconstruct afterwards.

3.2 Chunking

Chunking is the single most underrated lever in RAG quality. Embedding a whole 40-page PDF as one vector blurs everything together; embedding single sentences loses context. You want chunks that each express **one coherent idea**.

Strategy	How it works	Use when
Fixed-size	Cut every N characters/tokens	Quick baseline; uniform text
Recursive character	Try to split on paragraphs, then lines, then sentences, then words until chunks fit	**Default choice** for most prose (used in both projects)
Structure-aware	Split on Markdown/HTML headers, code functions, table rows	Docs with clear structure; keep header text in metadata
Semantic	Split where embedding similarity between sentences drops	Long, topic-shifting text; costs more
Parent-child	Index small child chunks, return the larger parent for context	Need precise matching but rich context

Parameter	Typical range	Trade-off
chunk_size	200-1000 tokens (about 800-4000 characters)	Small = precise but fragmentary. Large = more context but diluted vectors and more prompt tokens.
chunk_overlap	10-20% of chunk_size	Prevents losing facts at boundaries; too much wastes storage.
top-k	3-8	Higher k raises recall but adds noise and cost; pair with re-ranking.

> **TIP**
> There is no universally best chunk size. Build a small test set (Section 3.8), change one parameter at a time and measure hit rate. Project 1 ships a ready-made `eval_retrieval.py` for exactly this.

3.3 Embeddings

An embedding model maps text to a vector (for example 1536 numbers). The geometry encodes meaning: 'annual leave' and 'paid vacation days' land close together even though they share no words. This is why semantic search finds answers that keyword search misses.

Similarity metrics. The most common is *cosine similarity*: cos(a, b) = (a . b) / (|a| x |b|), the cosine of the angle between two vectors (1 = same direction, 0 = unrelated). For normalised vectors, dot

product and cosine give the same ranking; Euclidean (L2) distance also agrees. Chroma defaults to L2 on its own embeddings, which is fine for most models; set the collection metric deliberately in larger systems.

Model	Type	Dimensions	Notes
OpenAI text-embedding-3-small	API	1536 (adjustable)	Cheap, strong default; used by default here
OpenAI text-embedding-3-large	API	3072 (adjustable)	Higher quality, higher cost
sentence-transformers all-MiniLM-L6-v2	Local (free)	384	Small and fast; used in the 'local' provider
BGE / E5 / GTE families	Local or hosted	384-1024	Strong open models; check the MTEB leaderboard
Cohere / Voyage / Google	API	varies	Multilingual and domain-specific options

WARNING
Use the **same embedding model** (and version) for indexing and for queries. If you change the model you must re-index everything, because vectors from different models live in different spaces.

3.4 Vector databases

A vector database stores vectors with their text and metadata and answers 'give me the k nearest vectors' quickly. Exact search compares against every vector; real systems use **approximate nearest neighbour (ANN)** indexes such as **HNSW** (a layered proximity graph) or IVF (clustering) to trade a little accuracy for large speed-ups.

Store	Hosting	Good for
Chroma	Embedded / local, persistent folder	Learning, prototypes, small apps (used here)
FAISS	In-process library	Fast local search, research, no server
Qdrant / Weaviate / Milvus	Self-host or cloud	Production, filtering, scale
Pinecone	Managed cloud	Zero-ops production
pgvector (PostgreSQL)	Your existing Postgres	Keep vectors next to relational data

3.5 Retrieval strategies

Technique	Idea	When it helps
Similarity top-k	Return the k nearest chunks	Baseline
MMR (max marginal relevance)	Pick chunks that are relevant *and* different from each other	Results are near-duplicates
Score threshold	Drop chunks below a similarity cut-off	Avoid answering from irrelevant context
Metadata filter	Restrict by source, date, department, user permission	Multi-tenant or time-sensitive data
Hybrid search	Combine BM25 keyword scores with vector scores (often via Reciprocal Rank Fusion)	IDs, product codes, acronyms, rare words
Re-ranking	Retrieve 20-50 candidates, re-score with a cross-encoder, keep the best 3-5	Highest-impact quality upgrade

Technique	Idea	When it helps
Multi-query	LLM writes several paraphrases of the question; merge results	Ambiguous or short questions
HyDE	LLM writes a hypothetical answer; embed *that* for search	Question and document wording differ
Query rewriting / condensing	Turn follow-ups into standalone queries	Chatbots (used in Project 2)
Parent-document / compression	Return bigger context, or trim chunks to relevant sentences	Precise matching with rich context

Reciprocal Rank Fusion (RRF) merges ranked lists without needing comparable scores. Each document gets the sum of 1 / (k + rank) over every list it appears in (k is usually 60):

rrf.py

```
def reciprocal_rank_fusion(rank_lists, k=60):
    """rank_lists: e.g. [bm25_ids, vector_ids] -> fused list of ids, best first."""
    scores = {}
    for ranking in rank_lists:
        for rank, doc_id in enumerate(ranking, start=1):
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)

print(reciprocal_rank_fusion([["a", "b", "c"], ["c", "a", "d"]]))   # ['a', 'c', 'b', 'd']
```

3.6 Prompting for grounded generation

The prompt is where retrieval turns into a trustworthy answer. A good RAG prompt has five parts:

1. **Role and scope:** 'Answer using ONLY the context.'
2. **Refusal rule:** exact wording to use when the context lacks the answer (makes failures easy to detect).
3. **Citation rule:** number each chunk `[1]`, `[2]` and ask the model to cite numbers.
4. **Style rule:** concise, no invented details, same language as the question.
5. **Context then question:** chunks first, question last, so the question stays fresh in the model's attention.

> **NOTE**
> Models tend to use information at the **start and end** of a long context better than the middle ('lost in the middle'). Keep k small, put the best chunks first, and re-rank instead of stuffing 20 chunks.

Set **temperature to 0-0.2** for factual RAG. Creativity is a bug here.

3.7 Security and governance

- **Prompt injection through documents:** retrieved text may contain 'ignore previous instructions...'. Treat context as *data*; say so in the system prompt, and never give an LLM tools it does not need.
- **Access control:** filter by metadata (user, group, tenant) *at retrieval time*. Never rely on the LLM to hide documents the user should not see.
- **PII and secrets:** redact before indexing where possible; log carefully.
- **Freshness:** store an `updated_at` metadata field and re-index changed files; remove deleted files from the index.

3.8 Evaluating a RAG system

Never judge RAG by trying three questions and feeling good. Evaluate the two halves separately, because a wrong answer is caused either by *bad retrieval* or by *bad generation*, and the fixes differ.

Layer	Metric	Question it answers
Retrieval	**Hit rate / Recall@k**	Is a chunk containing the answer in the top-k?
Retrieval	**MRR** (mean reciprocal rank)	How high is the first correct chunk ranked? (1/rank, averaged)
Retrieval	Context precision	What fraction of retrieved chunks is actually relevant?
Generation	**Faithfulness / groundedness**	Is every claim supported by the retrieved context?
Generation	Answer relevance	Does the answer address the question asked?
End-to-end	Answer correctness	Does it match the reference answer?

Practical recipe: (1) write 30-100 realistic questions with the expected answer and the source passage; (2) measure hit rate@k first and fix retrieval until it is high; (3) then measure faithfulness and correctness, using an LLM-as-judge or a framework such as **RAGAS**, **TruLens** or **LangSmith** evaluations; (4) re-run the suite after every change to chunking, embeddings, prompts or models.

3.9 Common failure modes and fixes

Symptom	Likely cause	Fix
'I don't know' although the answer is in the files	Chunk split the answer; k too small; wrong embedding/query mismatch	Increase overlap/k, check chunks, add hybrid search or query rewriting
Confident but wrong answer	Irrelevant chunks retrieved, prompt allows outside knowledge	Stricter prompt, score threshold, document grading (Project 2), re-ranking
Answer mixes two products/years	Chunks lack identifying context	Prepend title/section to each chunk; metadata filters
Numbers from tables are wrong	PDF table extraction scrambled	Use a layout-aware parser; convert tables to Markdown/CSV
Follow-up question fails ('and for Pro?')	Retriever sees only the last message	Condense question with chat history (Project 2)
Slow or expensive	Too many chunks, big model, many LLM calls	Lower k, re-rank, cache, smaller model for grading
Good in testing, bad in production	Test questions unrepresentative	Log real queries; build the eval set from them

4. LangChain essentials you need for RAG

LangChain is a toolkit of interchangeable components. Five ideas cover almost everything in this manual.

4.1 Document

```
from langchain_core.documents import Document

doc = Document(page_content="Employees get 24 days of leave.",
               metadata={"source": "handbook.txt", "page": 0})
```

4.2 Runnable and the pipe operator (LCEL)

Prompts, models, parsers, retrievers and whole chains all implement the **Runnable** interface: `invoke` (one input), `batch` (many inputs in parallel), `stream` (incremental output) and async versions. The pipe operator `|` chains runnables: the output of the left one is the input of the right one. This is **LangChain Expression Language (LCEL)**.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer in one sentence."),
    ("human", "{question}"),
])
chain = prompt | llm | StrOutputParser()      # prompt -> model -> plain string
print(chain.invoke({"question": "What is RAG?"}))
```

4.3 RunnablePassthrough and RunnableParallel

- `RunnableParallel(a=x, b=y)` runs several runnables on the same input and returns a dict `{a: ..., b: ...}`.
- `RunnablePassthrough()` forwards the input unchanged (to keep the question next to the retrieved docs).
- `RunnablePassthrough.assign(key=runnable)` adds a new key to the incoming dict - used three times in Project 1.

4.4 Retriever

A retriever is a Runnable that maps a query string to `List[Document]`. Any vector store creates one with `vectorstore.as_retriever(search_type=..., search_kwargs={...})`.

4.5 Structured output

`llm.with_structured_output(MyPydanticModel)` makes the model return a validated Python object instead of free text. It is how Project 2 turns an LLM into a reliable *router* and *grader*.

```
from typing import Literal
from pydantic import BaseModel

class Verdict(BaseModel):
    binary_score: Literal["yes", "no"]

grader = prompt | llm.with_structured_output(Verdict)
grader.invoke({"document": "...", "query": "..."}).binary_score    # -> "yes"
```

5. Environment setup

5.1 Prerequisites

- Python **3.10 or newer** (3.11 / 3.12 recommended) and `pip`.
- Either an **OpenAI API key** (default) **or** a free local setup with **Ollama** (chat model) plus HuggingFace sentence-transformers (embeddings). The code switches with one variable: `LLM_PROVIDER=openai|local`.
- A terminal and any editor (VS Code recommended). About 2 GB free disk if you use local models.

5.2 Create the project and virtual environment

terminal

```
mkdir rag_project
cd rag_project
python -m venv .venv                # macOS/Linux: python3 -m venv .venv

.venv\Scripts\Activate.ps1           # Windows PowerShell
source .venv/bin/activate           # macOS / Linux

pip install -r requirements.txt     # the file is listed with each project
copy .env.example .env              # Windows;  macOS/Linux: cp .env.example .env
```

5.3 Choosing a provider

	openai (default)	local
Chat model	gpt-4o-mini (any chat model works)	Ollama: `ollama pull llama3.1`
Embeddings	text-embedding-3-small	all-MiniLM-L6-v2 via sentence-transformers (downloads once)
Cost	Pay per token (very small for this manual)	Free; needs RAM/CPU/GPU
Quality / speed	High / fast	Good with 8B models; slower on CPU
Set in .env	LLM_PROVIDER=openai and OPENAI_API_KEY=...	LLM_PROVIDER=local (Ollama must be running)

WARNING
Never commit `.env` or API keys to Git. Add `.env`, `.venv/`, `chroma_db/` to `.gitignore`.

VERSION NOTE
Library APIs move quickly. The code in this manual targets the current LangChain 0.3 / 1.x and LangGraph 0.2+ package split (`langchain-core`, `langchain-community`, `langchain-chroma`, `langchain-openai`...). If an import fails after an upgrade, check the release notes at python.langchain.com and langchain-ai.github.io/langgraph, or pin the versions you tested.

PART 2

Project 1
RAG from Scratch

A complete, runnable question-answering system over your own documents
LangChain + Chroma + OpenAI (or local models)

6. Project 1 overview: a document Q&A assistant

You will build a command-line assistant that answers questions about PDF, TXT and Markdown files, cites its sources, and refuses to guess. It implements the naive RAG pipeline from Chapter 2 with clean, separate modules so every concept maps to one file.

6.1 What you will build

- **Indexing pipeline** (`ingest.py`): load files, split with `RecursiveCharacterTextSplitter`, embed, persist in Chroma.
- **RAG chain** (`rag_chain.py`): retriever -> context formatter -> prompt -> LLM -> string, written in LCEL.
- **CLI** (`main.py`): `ingest`, `ask` and `chat` commands.
- **Retrieval test** (`eval_retrieval.py`): measures hit rate on a small question set - your first evaluation harness.
- **Provider switch**: the same code runs on OpenAI or on a fully local Ollama + HuggingFace stack.

6.2 Folder structure

project layout

```
rag_project/
|-- .env                    # your secrets (copy from .env.example)
|-- .env.example
|-- requirements.txt
|-- config.py               # all settings in one place
|-- models.py               # get_llm(), get_embeddings()  (provider switch)
|-- vectorstore.py          # opens the persistent Chroma store
|-- ingest.py               # load -> split -> embed -> store
|-- rag_chain.py            # the RAG pipeline (LCEL)
|-- main.py                 # CLI: ingest / ask / chat
|-- eval_retrieval.py       # retrieval hit-rate test
|-- create_sample_data.py   # writes demo documents into data/
|-- data/                   # your documents go here (.pdf, .txt, .md)
`-- chroma_db/              # created automatically by ingest
```

6.3 Build order (follow along)

1. Create the folder, virtual environment and install `requirements.txt` (Chapter 5).
2. Create each file below exactly as listed (copy-paste works).
3. Run `python create_sample_data.py` to generate demo documents - or put your own PDFs in `data/`.
4. Run `python main.py ingest`, then `python main.py chat`.
5. Run `python eval_retrieval.py` to see your retrieval hit rate.

7. Project 1 - source code

7.1 requirements.txt

requirements.txt

```
# --- core (both projects) ---
langchain-core>=0.3
langchain-community>=0.3
langchain-text-splitters>=0.3
langchain-chroma>=0.2
chromadb>=0.5
pypdf>=4.0
python-dotenv>=1.0
pydantic>=2.0

# --- OpenAI provider (default) ---
langchain-openai>=0.2

# --- local provider (LLM_PROVIDER=local): Ollama + HuggingFace embeddings ---
langchain-ollama>=0.2
langchain-huggingface>=0.1
sentence-transformers>=3.0
```

7.2 .env.example (copy to .env)

.env.example

```
# Copy this file to ".env" and fill in your values.
# Provider: "openai" (default) or "local" (Ollama chat model + HuggingFace embeddings)
LLM_PROVIDER=openai

# --- OpenAI ---
OPENAI_API_KEY=sk-your-key-here
OPENAI_CHAT_MODEL=gpt-4o-mini
OPENAI_EMBED_MODEL=text-embedding-3-small

# --- Local (only used when LLM_PROVIDER=local) ---
OLLAMA_CHAT_MODEL=llama3.1
HF_EMBED_MODEL=sentence-transformers/all-MiniLM-L6-v2
```

7.3 config.py

All tunable numbers live here, so experiments (chunk size, k, models) never touch the logic.

config.py

```
"""Central configuration. Values can be overridden with environment variables / .env."""
import os

from dotenv import load_dotenv

load_dotenv()

# --- provider ---------------------------------------------------------------
PROVIDER = os.getenv("LLM_PROVIDER", "openai").lower()  # "openai" or "local"

OPENAI_CHAT_MODEL = os.getenv("OPENAI_CHAT_MODEL", "gpt-4o-mini")
OPENAI_EMBED_MODEL = os.getenv("OPENAI_EMBED_MODEL", "text-embedding-3-small")
OLLAMA_CHAT_MODEL = os.getenv("OLLAMA_CHAT_MODEL", "llama3.1")
HF_EMBED_MODEL = os.getenv("HF_EMBED_MODEL", "sentence-transformers/all-MiniLM-L6-v2")

# --- data & index -----------------------------------------------------------
DATA_DIR = "data"
CHROMA_DIR = "chroma_db"
COLLECTION = "knowledge_base"

# --- chunking & retrieval ---------------------------------------------------
CHUNK_SIZE = 600       # characters per chunk
CHUNK_OVERLAP = 100    # characters shared between neighbouring chunks
TOP_K = 4              # chunks passed to the LLM
```

7.4 models.py

A tiny factory. Imports are done *inside* the branches so you only need the packages of the provider you use.

models.py

```
"""Model factory: returns the chat model and embedding model for the chosen provider."""
import config


def get_llm(temperature: float = 0.0):
    """Chat model used for answering, routing and grading."""
    if config.PROVIDER == "local":
        from langchain_ollama import ChatOllama

        return ChatOllama(model=config.OLLAMA_CHAT_MODEL, temperature=temperature)

    from langchain_openai import ChatOpenAI

    return ChatOpenAI(model=config.OPENAI_CHAT_MODEL, temperature=temperature)


def get_embeddings():
    """Embedding model. The SAME model must be used for indexing and querying."""
    if config.PROVIDER == "local":
        from langchain_huggingface import HuggingFaceEmbeddings

        return HuggingFaceEmbeddings(model_name=config.HF_EMBED_MODEL)

    from langchain_openai import OpenAIEmbeddings

    return OpenAIEmbeddings(model=config.OPENAI_EMBED_MODEL)
```

7.5 vectorstore.py

vectorstore.py

```
"""Single place that knows how to open the persistent Chroma vector store."""
from langchain_chroma import Chroma

import config
from models import get_embeddings


def get_vectorstore() -> Chroma:
    return Chroma(
        collection_name=config.COLLECTION,
        embedding_function=get_embeddings(),
        persist_directory=config.CHROMA_DIR,
    )


def count_chunks() -> int:
    """Number of chunks currently stored (0 means you forgot to run ingestion)."""
    return get_vectorstore()._collection.count()
```

7.6 ingest.py - the indexing pipeline

ingest.py

```
"""Indexing pipeline: load files -> split into chunks -> embed -> store in Chroma."""
import shutil
import sys
from pathlib import Path
from typing import List

from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader, TextLoader
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter

import config
from models import get_embeddings


def load_documents(data_dir: str) -> List[Document]:
    """Load every .pdf / .txt / .md file found under data_dir."""
    docs: List[Document] = []
    for path in sorted(Path(data_dir).rglob("*")):
        suffix = path.suffix.lower()
        if suffix == ".pdf":
            docs.extend(PyPDFLoader(str(path)).load())  # one Document per page
        elif suffix in {".txt", ".md"}:
            docs.extend(TextLoader(str(path), encoding="utf-8").load())
    return docs


def split_documents(docs: List[Document]) -> List[Document]:
    """Cut documents into overlapping chunks, keeping metadata (source, page)."""
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=config.CHUNK_SIZE,
        chunk_overlap=config.CHUNK_OVERLAP,
        separators=["\n\n", "\n", ". ", " ", ""],
        add_start_index=True,
    )
    return splitter.split_documents(docs)


def build_index(rebuild: bool = True) -> int:
    """Create (or recreate) the vector store. Returns the number of chunks indexed."""
    if rebuild and Path(config.CHROMA_DIR).exists():
        shutil.rmtree(config.CHROMA_DIR)

    docs = load_documents(config.DATA_DIR)
    if not docs:
        sys.exit(f"No .pdf/.txt/.md files found in '{config.DATA_DIR}/'. Add some first.")

    chunks = split_documents(docs)
    print(f"Loaded {len(docs)} document(s) -> {len(chunks)} chunk(s). Embedding...")

    Chroma.from_documents(
        documents=chunks,
        embedding=get_embeddings(),
        collection_name=config.COLLECTION,
        persist_directory=config.CHROMA_DIR,
    )
    print(f"Index saved in '{config.CHROMA_DIR}/'.")
    return len(chunks)


if __name__ == "__main__":
    build_index()
```

7.7 rag_chain.py - the heart of the project

rag_chain.py

```
"""The RAG chain, written with LangChain Expression Language (LCEL)."""
from pathlib import Path
from typing import Dict, List

from langchain_core.documents import Document
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import Runnable, RunnableParallel, RunnablePassthrough

import config
from models import get_llm
from vectorstore import count_chunks, get_vectorstore

SYSTEM_PROMPT = (
    "You are a precise assistant that answers questions using ONLY the context provided.\n"
    "Rules:\n"
    "1. If the context does not contain the answer, reply exactly: "
    "\"I don't know based on the provided documents.\"\n"
    "2. Be concise and factual. Never invent details.\n"
    "3. Cite the sources you used by their numbers, for example [1] or [2]."
)

PROMPT = ChatPromptTemplate.from_messages(
    [
        ("system", SYSTEM_PROMPT),
        ("human", "Context:\n{context}\n\nQuestion: {question}"),
    ]
)


def source_label(doc: Document) -> str:
    """Human-readable origin of a chunk, e.g. 'handbook.pdf, page 3'."""
    name = Path(doc.metadata.get("source", "unknown")).name
    page = doc.metadata.get("page")
    return f"{name}, page {page + 1}" if page is not None else name


def format_docs(docs: List[Document]) -> str:
    """Turn retrieved chunks into one numbered context block for the prompt."""
    return "\n\n".join(
        f"[{i}] ({source_label(d)})\n{d.page_content}" for i, d in enumerate(docs, start=1)
    )


def build_rag_chain() -> Runnable:
    """Input: a question string.  Output: dict with docs, question, context, answer."""
    if count_chunks() == 0:
        raise RuntimeError("The vector store is empty. Run:  python main.py ingest")

    retriever = get_vectorstore().as_retriever(
        search_type="similarity",
        search_kwargs={"k": config.TOP_K},
    )
    generate = PROMPT | get_llm() | StrOutputParser()

    return (
        # 1) run the retriever and keep the original question, in parallel
        RunnableParallel(docs=retriever, question=RunnablePassthrough())
        # 2) build the context string from the retrieved documents
        | RunnablePassthrough.assign(context=lambda x: format_docs(x["docs"]))
        # 3) call the LLM with prompt(context, question)
        | RunnablePassthrough.assign(answer=generate)
    )


def ask(chain: Runnable, question: str) -> Dict:
    """Run the chain and return the answer together with de-duplicated sources."""
    result = chain.invoke(question)
    sources = list(dict.fromkeys(source_label(d) for d in result["docs"]))
    return {"answer": result["answer"], "sources": sources, "docs": result["docs"]}
```

7.8 main.py - command-line interface

main.py

```
"""Command-line interface for Project 1 (plain RAG).

Usage:
    python create_sample_data.py          # optional: writes demo files into data/
    python main.py ingest                 # build the vector index
    python main.py ask "How many days of annual leave do I get?"
    python main.py chat                   # interactive loop
"""
import argparse

from ingest import build_index
from rag_chain import ask, build_rag_chain


def print_result(result: dict) -> None:
    print(f"\nAssistant: {result['answer']}")
    print("Sources  :", "; ".join(result["sources"]) or "-")


def chat() -> None:
    chain = build_rag_chain()
    print("RAG chat ready. Type 'exit' to quit.")
    while True:
        try:
            question = input("\nYou: ").strip()
        except (EOFError, KeyboardInterrupt):
            print()
            break
        if question.lower() in {"exit", "quit"}:
            break
        if question:
            print_result(ask(chain, question))


def main() -> None:
    parser = argparse.ArgumentParser(description="Simple RAG over your documents")
    sub = parser.add_subparsers(dest="command", required=True)
    sub.add_parser("ingest", help="(re)build the vector index from data/")
    ask_cmd = sub.add_parser("ask", help="ask a single question")
    ask_cmd.add_argument("question")
    sub.add_parser("chat", help="interactive question loop")
    args = parser.parse_args()

    if args.command == "ingest":
        build_index()
    elif args.command == "ask":
        print_result(ask(build_rag_chain(), args.question))
    else:
        chat()


if __name__ == "__main__":
    main()
```

7.9 create_sample_data.py - demo documents

Fictional company facts, so you can verify every answer by eye.

create_sample_data.py

```
"""Writes two small, fictional documents into data/ so you can test the projects."""
from pathlib import Path

HANDBOOK = """NovaTech Solutions - Employee Handbook (2026 Edition)

1. Working Hours
Standard working time is 40 hours per week. Core hours are 10:00 to 16:00 local time,
during which employees are expected to be available for meetings.

2. Remote Work
Employees may work from home up to 3 days per week. Fully remote arrangements require
written approval from a Vice President. Remote workers must use the company VPN.

3. Leave Policy
Every employee receives 24 days of paid annual leave per calendar year, plus 10 public
holidays. Up to 5 unused leave days may be carried over into the next year.
Sick leave is 12 days per year; a medical certificate is required after 3 consecutive days.
Parental leave is 26 weeks for the primary caregiver and 8 weeks for the secondary caregiver.

4. Travel and Expenses
Meals during business travel are reimbursed up to USD 60 per day. Flights shorter than
6 hours must be booked in economy class. Expense claims must be submitted within 30 days
through the NovaPay portal, attaching all receipts.

5. Learning Budget
Each employee has a learning budget of USD 1,200 per year for courses, books and
conferences. Unused budget does not roll over.

6. Information Security
All laptops use full-disk encryption and two-factor authentication is mandatory.
Any suspected security incident must be reported within 2 hours to the security team
at security@novatech.example.
"""

FAQ = """# NovaDesk Product FAQ

NovaDesk is NovaTech's customer help-desk software.

## Pricing and plans
- Starter: USD 12 per user per month.
- Pro: USD 29 per user per month.
- Enterprise: custom pricing, contact sales.
Every plan includes a 14-day free trial. Annual plans have a 30-day money-back guarantee.

## Support
Standard support is available Monday to Friday, 08:00 to 20:00 UTC.
Enterprise customers receive 24/7 support with a dedicated account manager.

## Reliability
Pro and Enterprise plans include a 99.9% uptime SLA. Starter has no SLA.

## Data retention
Tickets are stored for 24 months on Starter, 60 months on Pro, and a custom period on Enterprise.

## API and integrations
The API rate limit is 100 requests per minute on Starter and 1,000 requests per minute on Pro.
NovaDesk integrates with Slack, Jira and Salesforce.
"""


def main() -> None:
    data = Path("data")
    data.mkdir(exist_ok=True)
    (data / "novatech_handbook.txt").write_text(HANDBOOK, encoding="utf-8")
    (data / "novadesk_faq.md").write_text(FAQ, encoding="utf-8")
    print("Sample files written to data/")


if __name__ == "__main__":
    main()
```

7.10 eval_retrieval.py - measure retrieval quality

eval_retrieval.py

```
"""Tiny retrieval evaluation: does the right chunk appear in the top-k results?

Each test case lists a question and a keyword that MUST appear in at least one
retrieved chunk. Hit rate = (# questions answered by retrieval) / (# questions).
"""
import config
from vectorstore import get_vectorstore

TEST_CASES = [
    ("How many days of annual leave do employees get?", "24 days"),
    ("How many days can I work from home?", "3 days per week"),
    ("What is the meal allowance when travelling?", "USD 60"),
    ("How long do I have to submit expense claims?", "30 days"),
    ("What does the Pro plan cost?", "USD 29"),
    ("What is the API rate limit on Starter?", "100 requests per minute"),
    ("How long are tickets stored on the Pro plan?", "60 months"),
    ("Who do I report a security incident to?", "security@novatech.example"),
]


def main() -> None:
    retriever = get_vectorstore().as_retriever(search_kwargs={"k": config.TOP_K})
    hits = 0
    for question, keyword in TEST_CASES:
        docs = retriever.invoke(question)
        found = any(keyword.lower() in d.page_content.lower() for d in docs)
        hits += found
        print(f"[{'HIT ' if found else 'MISS'}] {question}")
    rate = hits / len(TEST_CASES)
    print(f"\nHit rate @ k={config.TOP_K}: {hits}/{len(TEST_CASES)} = {rate:.0%}")


if __name__ == "__main__":
    main()
```

8. Project 1 - run it and understand it

8.1 Running

terminal

```
python create_sample_data.py          # writes data/novatech_handbook.txt and data/novadesk_faq.md
python main.py ingest                 # Loaded 2 document(s) -> N chunk(s). Embedding...
python main.py ask "How many days of annual leave do I get?"
python main.py chat
python eval_retrieval.py
```

Illustrative session (your exact wording will differ slightly by model):

example chat

```
You: How many days of annual leave do I get, and can I carry any over?

Assistant: Employees receive 24 days of paid annual leave per calendar year plus 10 public
holidays, and up to 5 unused days can be carried over into the next year. [1]
Sources  : novatech_handbook.txt

You: What is the CEO's favourite colour?

Assistant: I don't know based on the provided documents.
Sources  : novatech_handbook.txt; novadesk_faq.md
```

> **NOTE**
> The second answer is the *correct* behaviour: the documents contain nothing about it, so the system refuses instead of hallucinating. Sources are still listed because they were *retrieved*, not necessarily *used* - Project 2 fixes this with document grading.

8.2 Test questions to try

Question	Expected answer (from the sample data)
How many days can I work from home?	Up to 3 days per week; fully remote needs VP approval
What is the daily meal allowance on business trips?	USD 60 per day
How long is parental leave for the secondary caregiver?	8 weeks
Which plan has a 99.9% uptime SLA?	Pro and Enterprise
What is the API rate limit for the Pro plan?	1,000 requests per minute
Is there a money-back guarantee?	Yes - 30 days on annual plans
What is the capital of France?	I don't know based on the provided documents.

8.3 How the code works - key lines explained

Code	What happens
`RecursiveCharacterTextSplitter(chunk_size=600, chunk_overlap=100)`	Splits on paragraph, line, sentence, word boundaries (in that order) so chunks end at natural places. `add_start_index=True` stores each chunk's character offset in metadata.
`Chroma.from_documents(...)`	Embeds every chunk (one API call per batch) and writes vectors + text + metadata into `chroma_db/`. Re-running `ingest` wipes and rebuilds the index so you never get duplicates.

Code	What happens
`as_retriever(search_kwargs={'k': 4})`	Wraps the vector store as a Runnable: question string in, 4 `Document`s out.
`RunnableParallel(docs=retriever, question=RunnablePassthrough())`	Runs retrieval while carrying the original question forward. Result: `{docs: [...], question: '...'}`.
`RunnablePassthrough.assign(context=...)`	Adds `context` = numbered chunk text to the dict.
`RunnablePassthrough.assign(answer=generate)`	Runs `prompt | llm | parser` on the dict and adds `answer`. The final dict keeps `docs` so the CLI can print sources.
`count_chunks() == 0` guard	Fails early with a friendly message if you forgot to run `ingest`.

8.4 Experiments that teach the most

1. **Chunk size:** set `CHUNK_SIZE` to 200, then 1500. Re-ingest and re-run `eval_retrieval.py`. Which hit rate is best?
2. **Top-k:** set `TOP_K` to 1, 2, 8. Watch hit rate (up) and answer noise (also up).
3. **Prompt:** remove rule 1 from `SYSTEM_PROMPT` and ask the capital-of-France question. Observe hallucination return - this shows why the refusal rule exists.
4. **MMR:** change `search_type` to `'mmr'` with `search_kwargs={'k': 4, 'fetch_k': 20}`.
5. **Your own PDFs:** drop them into `data/`, re-ingest, and write 10 test questions for `eval_retrieval.py`.
6. **Local models:** `ollama pull llama3.1`, set `LLM_PROVIDER=local`, re-ingest (new embeddings!) and compare quality.

8.5 Troubleshooting

Problem	Fix
`AuthenticationError` / 401	Check `OPENAI_API_KEY` in `.env`; restart the terminal after editing.
'The vector store is empty'	Run `python main.py ingest` first (and make sure `data/` has files).
Switched provider and results are garbage / dimension error	Embeddings differ per model. Delete `chroma_db/` (or re-run `ingest`, which rebuilds).
`ModuleNotFoundError`	Activate the virtual environment and re-run `pip install -r requirements.txt`.
Ollama connection refused	Start Ollama (`ollama serve`) and make sure the model is pulled.
Scanned PDF returns empty text	It has no text layer; run OCR first (e.g. `ocrmypdf`).
Answers cite the wrong file	Inspect `result['docs']`; lower chunk size or add metadata filters.

8.6 Upgrade ideas (adds up to a portfolio project)

- Streaming output with `chain.stream(...)` and a Streamlit or Gradio front end.
- Hybrid search (BM25 + vectors) and a cross-encoder re-ranker.
- Metadata filters, e.g. `search_kwargs={'k': 4, 'filter': {'source': 'novadesk_faq.md'}}`.
- Incremental indexing: hash each file and only re-embed changed files.
- Chat history: condense follow-ups into standalone questions - built in Project 2.

PART 3

LangGraph Crash Course

State, nodes, edges, conditional routing, memory, tools, streaming and agentic RAG patterns

9. Why LangGraph?

A **chain** is a fixed line: A then B then C. Real assistants need more: *branch* ('is this question about our documents?'), *loop* ('the retrieval was poor - rewrite the query and try again'), *remember* the conversation, *pause* for a human, and *recover* after a crash. Writing that with nested `if` statements and while-loops gets messy fast.

LangGraph is a low-level orchestration library (from the LangChain team) that models an application as a **graph**: a shared **state**, **nodes** that read and update it, and **edges** that decide what runs next. Loops and branches are first-class, and the runtime gives you persistence (checkpoints), streaming and human-in-the-loop for free.

	Chain (LCEL)	Agent (LLM picks tools)	LangGraph workflow
Control flow	Fixed sequence	LLM decides every step	You define the graph; LLM may decide at chosen points
Loops / branches	Awkward	Implicit (inside the agent)	Explicit and visible
Predictability	Highest	Lowest	High - you set the guard rails
Memory / resume	Manual	Manual	Built-in checkpointers
Best for	Simple pipelines	Open-ended tasks	Production agents and agentic RAG

TIP
Rule of thumb: start with a chain. Move to LangGraph when you need a **loop**, a **conditional branch**, **memory across turns**, or **human approval**.

10. Core concepts

Concept	What it is	In code
State	The shared data the whole graph reads and writes. A schema: `TypedDict`, dataclass or Pydantic model.	`class State(TypedDict): ...`
Node	A Python function `(state) -> partial update`. Does the work: call an LLM, retrieve, run a tool.	`builder.add_node('name', fn)`
Edge	A fixed transition: after node A always run node B.	`builder.add_edge('A', 'B')`
Conditional edge	A routing function reads the state and returns the *name* of the next node.	`builder.add_conditional_edges('A', router, mapping)`
START / END	Special virtual nodes marking entry and exit.	`from langgraph.graph import START, END`
Reducer	A rule for *merging* an update into a state key (append instead of overwrite).	`Annotated[list, add_messages]`
Compile	Validates the graph and returns a runnable app; attach a checkpointer here.	`graph = builder.compile()`
Checkpointer	Saves the state after each step, keyed by `thread_id`: memory, resume, time travel.	`compile(checkpointer=MemorySaver())`

10.1 How execution works

1. You call `graph.invoke(initial_state)`. The input is written into the state.
2. The runtime runs the node(s) reachable from START. Each node returns a **dict containing only the keys it changes**.
3. The runtime merges the dict into the state: keys without a reducer are **overwritten**; keys with a reducer are **combined** (e.g. messages appended).
4. Edges (fixed or conditional) decide the next node(s). Nodes that become ready together run in the same 'super-step' (in parallel).
5. When END is reached, the final state is returned. If a loop never reaches END, a **recursion limit** stops it with an error - always set one.

> **KEY IDEA**
> Nodes do **not** mutate the state and do **not** call each other. They return updates; the graph decides what happens next. This is what makes graphs testable and resumable.

10.2 Reducers: overwrite vs append

```
import operator
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

class State(TypedDict):
    count: int                                  # no reducer  -> new value REPLACES old value
    log: Annotated[list[str], operator.add]     # operator.add -> new list is APPENDED to old list
    messages: Annotated[list, add_messages]     # chat reducer: appends by message id
```

`MessagesState` is a ready-made state with a single `messages` key using `add_messages`. Extend it when you need extra keys: `class State(MessagesState): documents: list`.

11. Your first graph (no LLM needed)

This example uses no model, so you can run it immediately and see the mechanics: a node that increments a counter, and a conditional edge that loops until the counter reaches 3.

hello_graph.py

```
import operator
from typing import Annotated, Literal
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END


class State(TypedDict):
    count: int
    log: Annotated[list[str], operator.add]


def increment(state: State):
    new = state["count"] + 1
    return {"count": new, "log": [f"count is now {new}"]}      # partial update


def should_continue(state: State) -> Literal["again", "stop"]:
    return "again" if state["count"] < 3 else "stop"           # routing function


builder = StateGraph(State)
builder.add_node("increment", increment)
builder.add_edge(START, "increment")
builder.add_conditional_edges("increment", should_continue,
                              {"again": "increment", "stop": END})
graph = builder.compile()

print(graph.invoke({"count": 0, "log": []}))
# {'count': 3, 'log': ['count is now 1', 'count is now 2', 'count is now 3']}

print(graph.get_graph().draw_mermaid())      # paste the output into mermaid.live to see the diagram
```

Read it as a story: START -> `increment` -> router says 'again' -> `increment` -> ... -> router says 'stop' -> END. `count` was overwritten each time; `log` accumulated because it has the `operator.add` reducer.

11.1 Visualising and debugging

- `graph.get_graph().draw_mermaid()` prints Mermaid text; `draw_mermaid_png()` renders an image (may need internet).
- `graph.stream(input, stream_mode='updates')` prints each node's output as it finishes - the quickest debugger.
- **LangSmith** tracing (`LANGSMITH_TRACING=true` plus an API key) records every node, prompt and token.
- **LangGraph Studio** (the developer UI) lets you step through, edit state and replay runs.

12. A chatbot with memory

Add an LLM node and a **checkpointer**. The checkpointer stores the state after every step under a `thread_id`; calling the graph again with the same id continues the same conversation.

chatbot_memory.py

```
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.checkpoint.memory import MemorySaver
from langchain_core.messages import HumanMessage
from models import get_llm

llm = get_llm()

def chatbot(state: MessagesState):
    return {"messages": [llm.invoke(state["messages"])]}      # reducer appends the AI reply

builder = StateGraph(MessagesState)
builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)
graph = builder.compile(checkpointer=MemorySaver())

cfg = {"configurable": {"thread_id": "alice-1"}}
graph.invoke({"messages": [HumanMessage("Hi, my name is Priya.")]}, cfg)
out = graph.invoke({"messages": [HumanMessage("What is my name?")]}, cfg)
print(out["messages"][-1].content)            # -> "Your name is Priya."

other = {"configurable": {"thread_id": "bob-1"}}               # a different thread = fresh memory
```

Checkpointer	Storage	Use for
`MemorySaver` (InMemorySaver)	Process RAM	Learning, tests (lost on restart)
`SqliteSaver`	SQLite file	Local apps, single machine
`PostgresSaver`	PostgreSQL	Production, multiple workers

Useful calls: `graph.get_state(cfg)` shows the latest state; `graph.get_state_history(cfg)` lists every checkpoint (time travel); `graph.update_state(cfg, {...})` edits the state manually.

13. Tools and the agent loop

A **tool** is a Python function the LLM may ask to call. The classic agent loop is a two-node graph: the **assistant** node (LLM) either answers or requests a tool; the **tools** node executes the request and feeds the result back. A conditional edge decides between 'call tools' and 'finish'.

tool_agent.py

```
from langchain_core.tools import tool
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.prebuilt import ToolNode, tools_condition
from models import get_llm


@tool
def multiply(a: int, b: int) -> int:
    """Multiply two integers."""
    return a * b


@tool
def search_docs(query: str) -> str:
    """Search the company knowledge base and return relevant passages."""
    return "Employees get 24 days of paid annual leave."        # plug in your retriever here


tools = [multiply, search_docs]
llm_with_tools = get_llm().bind_tools(tools)                    # the model can now request tools


def assistant(state: MessagesState):
    return {"messages": [llm_with_tools.invoke(state["messages"])]}


builder = StateGraph(MessagesState)
builder.add_node("assistant", assistant)
builder.add_node("tools", ToolNode(tools))                      # runs the requested tool calls
builder.add_edge(START, "assistant")
builder.add_conditional_edges("assistant", tools_condition)     # -> "tools" if tool calls, else END
builder.add_edge("tools", "assistant")                          # loop back with the tool result
agent = builder.compile()

result = agent.invoke({"messages": [("user", "What is 12 times 7?")]})
print(result["messages"][-1].content)
```

TIP

Prebuilt shortcuts exist: `langgraph.prebuilt.create_react_agent(llm, tools)` and, in newer LangChain releases, `langchain.agents.create_agent`. Learn the manual version first - it is exactly what these helpers build, and agentic RAG needs custom graphs anyway.

14. Streaming, human-in-the-loop and design rules

14.1 Stream modes

stream_mode	Yields	Use for
`values`	Full state after each step	Debugging, simple UIs
`updates`	Only the changes each node made, as `{node: update}`	Progress traces (used in Project 2)
`messages`	LLM tokens as they are generated	Typing effect in chat UIs
`custom`	Anything you emit with the stream writer	Progress bars, tool progress
`debug`	Detailed events	Deep troubleshooting

14.2 Human-in-the-loop

`interrupt()` pauses the graph inside a node and saves the state (a checkpointer is required). You resume later with `Command(resume=...)`; the value becomes the return value of `interrupt()`.

```
from langgraph.types import interrupt, Command

def approval(state):
    answer = interrupt({"question": "Send this email? (yes/no)", "draft": state["draft"]})
    return {"approved": answer == "yes"}

# ... build the graph with a checkpointer, then:
graph.invoke({"draft": "Hello..."}, cfg)          # runs until interrupt()
graph.invoke(Command(resume="yes"), cfg)          # continues from the same point
```

14.3 Parallel branches and sub-graphs (preview)

- Add two edges from one node to two different nodes: they run in the same super-step (**fan-out**). Give the shared state key a reducer so both results are kept (**fan-in**).
- The `Send` API creates a dynamic number of parallel tasks (map-reduce), e.g. summarise each of N documents.
- A compiled graph can be used as a node inside another graph (**sub-graph**) to keep large systems modular.

14.4 Ten design rules for reliable graphs

1. Keep **state small and explicit**; store what later nodes need, not everything.
2. One node = one job (retrieve, grade, rewrite, generate). Small nodes are easy to test.
3. Nodes return **partial updates**; never mutate the incoming state.
4. Put routing logic in **routing functions**, not inside nodes; make them pure and cheap.
5. Use **structured output** (Pydantic + `Literal`) for every LLM decision so routes are always valid.
6. **Bound every loop** with a counter in the state *and* a `recursion_limit` in the config.
7. Always end every path at a single `finalize` node when you must post-process output (as Project 2 does).
8. Use a **cheaper/faster model** for grading and routing, a stronger one for the final answer.
9. Log with LangSmith or `stream_mode='updates'` from day one.

10. Test nodes as plain Python functions with fake states before wiring the graph.

15. From naive RAG to agentic RAG with LangGraph

15.1 The simplest RAG graph (two nodes)

Plain RAG is a linear graph. This is the official 'hello world' of RAG in LangGraph and the bridge from Project 1:

simple_rag_graph.py

```
from typing import List
from typing_extensions import TypedDict
from langchain_core.documents import Document
from langgraph.graph import StateGraph, START, END


class RAGState(TypedDict):
    question: str
    context: List[Document]
    answer: str


def retrieve(state: RAGState):
    return {"context": retriever.invoke(state["question"])}


def generate(state: RAGState):
    docs_text = "\n\n".join(d.page_content for d in state["context"])
    reply = (prompt | llm).invoke({"context": docs_text, "question": state["question"]})
    return {"answer": reply.content}


builder = StateGraph(RAGState)
builder.add_node("retrieve", retrieve)
builder.add_node("generate", generate)
builder.add_edge(START, "retrieve")
builder.add_edge("retrieve", "generate")
builder.add_edge("generate", END)
rag_graph = builder.compile()

print(rag_graph.invoke({"question": "How many leave days?"})["answer"])
```

15.2 Three research patterns you should know

Pattern	Idea	Graph shape
Adaptive RAG	Route by question type/complexity: no retrieval, single-step retrieval, or multi-step research.	Conditional edge right after START
Corrective RAG (CRAG)	Grade retrieved documents. If they are poor, fix the query or fall back to another source (e.g. web search) *before* generating.	retrieve -> grade -> (generate | rewrite -> retrieve | fallback)
Self-RAG	After generating, self-check the answer for support by the documents and for usefulness; retry if it fails.	generate -> grade_generation -> (finish | retry)

Project 2 combines all three in one graph, adds conversation memory with question condensing, and guards every loop. The next part builds it step by step.

PART 4

Project 2
Agentic RAG with LangGraph

Routing - document grading - query rewriting - hallucination check
conversation memory - live node tracing

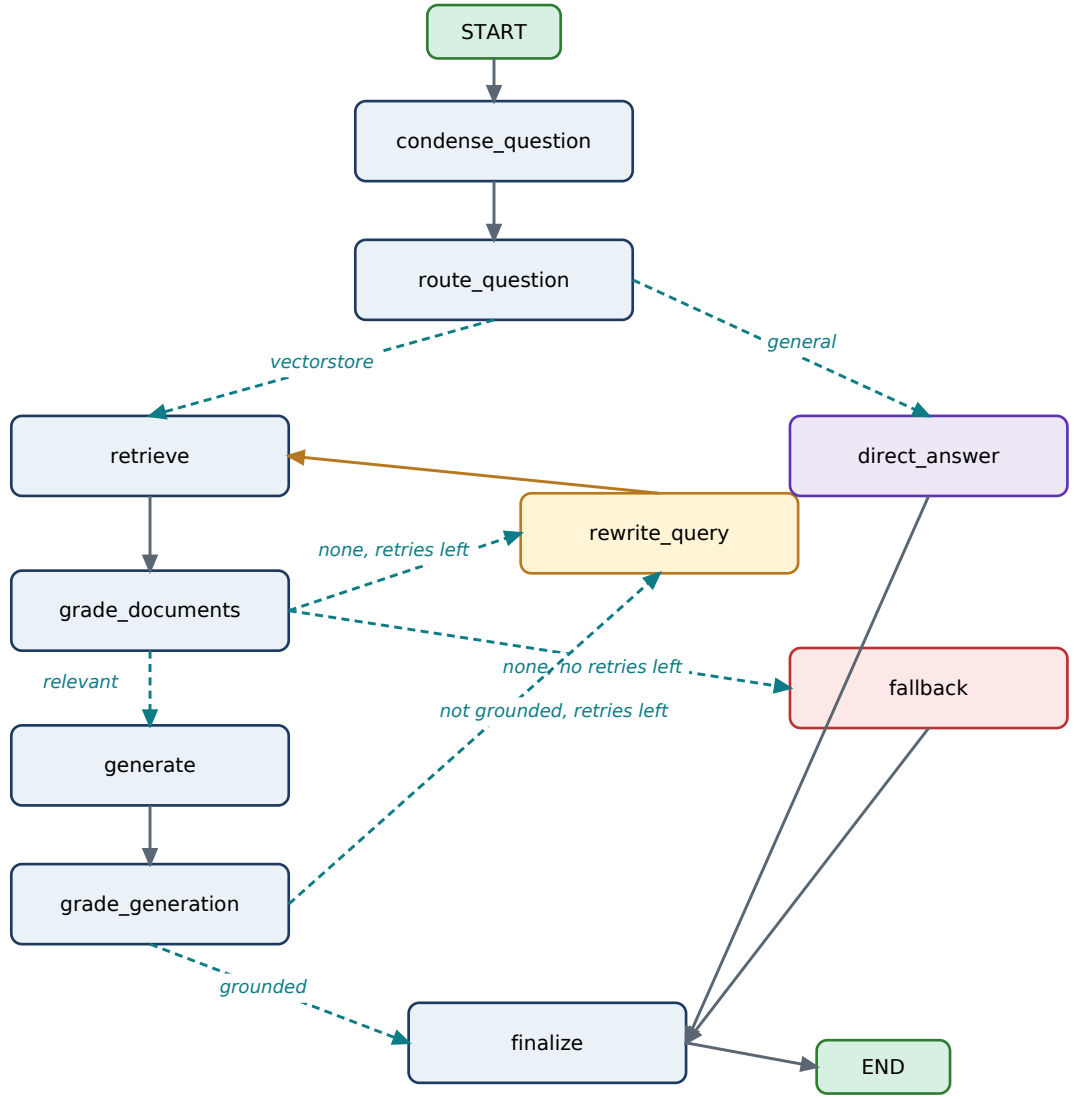
16. Project 2 overview: a self-correcting RAG agent

Project 1 always does the same thing: retrieve k chunks and answer, even if the chunks are irrelevant or the user just said 'hello'. Project 2 turns the pipeline into a **LangGraph** that makes decisions and repairs its own mistakes.

16.1 Features

Capability	Node(s)	Pattern
Understands follow-ups ('and for the Pro plan?')	`condense_question`	Conversational RAG
Decides whether to search the documents at all	`route_question`, `direct_answer`	Adaptive RAG
Filters out irrelevant chunks	`grade_documents`	Corrective RAG
Rewrites the query and retries when nothing relevant was found	`rewrite_query`	Corrective RAG
Checks the answer is supported by the documents	`grade_generation`	Self-RAG
Fails gracefully instead of hallucinating	`fallback`, `finalize`	Guard rails
Remembers the conversation	`MemorySaver` + `add_messages`	Persistence
Shows which nodes ran on every question	`stream_mode='updates'`	Observability

16.2 The graph



Dashed teal arrows = conditional edges (decided at run time). Solid = fixed edges.

16.3 Folder structure

project layout

```
agentic_rag/
|-- .env  /  .env.example
|-- requirements.txt
|-- config.py              # settings, MAX_RETRIES, KB_DESCRIPTION (used by the router)
|-- models.py              # get_llm(), get_embeddings()
|-- vectorstore.py         # opens the persistent Chroma store
|-- ingest.py              # indexing pipeline (same as Project 1)
|-- state.py               # GraphState: the shared state schema
|-- schemas.py             # Pydantic models for structured LLM decisions
|-- utils.py               # format_docs(), source_label()
|-- nodes.py               # every node + routing functions + prompts
|-- graph.py               # wires nodes and edges, compiles with a checkpointer
|-- main.py                # CLI: ingest / ask / chat / graph
|-- create_sample_data.py  # demo documents
|-- data/                  # your documents
`-- chroma_db/             # created by ingest
```

TIP

Shortcut: if you already built Project 1, copy its folder, then add `state.py`, `schemas.py`, `utils.py`, `nodes.py`, `graph.py`, replace `config.py` and `main.py`, and `pip install langgraph`. The vector index and sample data are fully reusable.

16.4 Design walk-through: what each node does

Node	Reads from state	Writes to state	LLM call?
`condense_question`	messages, question	query	Only if there is chat history
`route_question`	query	route	Yes - structured output `RouteQuery`
`retrieve`	query	documents	No (embedding search)
`grade_documents`	documents, query	documents (filtered)	Yes - one cheap call per chunk, run in parallel
`rewrite_query`	query, retries	query, retries+1	Yes
`generate`	documents, messages, question	generation	Yes
`grade_generation`	documents, generation	grounded	Yes - structured output `GradeGeneration`
`direct_answer`	messages, question	generation	Yes
`fallback`	-	generation (fixed text)	No
`finalize`	generation, grounded	generation, messages	No

Routing function	Returns	Meaning
`decide_route`	vectorstore / general	Search the documents or answer directly
`decide_after_grading`	generate / rewrite / fallback	Relevant chunks exist -> answer. None and retries left -> rewrite. None and no retries left -> give up politely
`decide_after_generation`	finalize / rewrite	Grounded (or retries exhausted) -> finish. Not grounded and retries left -> try retrieval again with a new query

17. Project 2 - source code

Create the files in this order. The first seven (requirements, `.env.example`, `config.py`, `models.py`, `vectorstore.py`, `ingest.py`, `create_sample_data.py`) are infrastructure and almost identical to Project 1; the LangGraph logic lives in `state.py`, `schemas.py`, `utils.py`, `nodes.py`, `graph.py` and `main.py`.

17.1 requirements.txt

requirements.txt

```
# --- core (both projects) ---
langchain-core>=0.3
langchain-community>=0.3
langchain-text-splitters>=0.3
langchain-chroma>=0.2
chromadb>=0.5
pypdf>=4.0
python-dotenv>=1.0
pydantic>=2.0

# --- OpenAI provider (default) ---
langchain-openai>=0.2

# --- local provider (LLM_PROVIDER=local): Ollama + HuggingFace embeddings ---
langchain-ollama>=0.2
langchain-huggingface>=0.1
sentence-transformers>=3.0

# --- LangGraph (Project 2) ---
langgraph>=0.2
```

17.2 .env.example (copy to .env)

.env.example

```
# Copy this file to ".env" and fill in your values.
# Provider: "openai" (default) or "local" (Ollama chat model + HuggingFace embeddings)
LLM_PROVIDER=openai

# --- OpenAI ---
OPENAI_API_KEY=sk-your-key-here
OPENAI_CHAT_MODEL=gpt-4o-mini
OPENAI_EMBED_MODEL=text-embedding-3-small

# --- Local (only used when LLM_PROVIDER=local) ---
OLLAMA_CHAT_MODEL=llama3.1
HF_EMBED_MODEL=sentence-transformers/all-MiniLM-L6-v2
```

17.3 config.py

Note the two new settings: `MAX_RETRIES` bounds the self-correction loops, and `KB_DESCRIPTION` tells the router what the knowledge base contains. **When you index different documents, edit `KB_DESCRIPTION`.**

config.py

```
"""Central configuration. Values can be overridden with environment variables / .env."""
import os

from dotenv import load_dotenv

load_dotenv()

# --- provider ----------------------------------------------------------
PROVIDER = os.getenv("LLM_PROVIDER", "openai").lower()  # "openai" or "local"

OPENAI_CHAT_MODEL = os.getenv("OPENAI_CHAT_MODEL", "gpt-4o-mini")
OPENAI_EMBED_MODEL = os.getenv("OPENAI_EMBED_MODEL", "text-embedding-3-small")
OLLAMA_CHAT_MODEL = os.getenv("OLLAMA_CHAT_MODEL", "llama3.1")
HF_EMBED_MODEL = os.getenv("HF_EMBED_MODEL", "sentence-transformers/all-MiniLM-L6-v2")

# --- data & index ------------------------------------------------------
DATA_DIR = "data"
CHROMA_DIR = "chroma_db"
COLLECTION = "knowledge_base"

# --- chunking & retrieval ----------------------------------------------
CHUNK_SIZE = 600
CHUNK_OVERLAP = 100
TOP_K = 4

# --- agentic behaviour -------------------------------------------------
MAX_RETRIES = 2        # how many times the graph may rewrite the query and retry
HISTORY_MESSAGES = 6   # how many previous chat messages are shown to the LLM

# Describes what is inside the vector store; the router uses it to decide
# between "vectorstore" and "general". Edit this when you change your documents!
KB_DESCRIPTION = (
    "the NovaTech Solutions employee handbook (working hours, remote work, leave, "
    "travel and expenses, learning budget, information security) and the NovaDesk "
    "product FAQ (pricing, plans, support hours, SLA, data retention, API limits, "
    "integrations)"
)
```

17.4 models.py

models.py

```
"""Model factory: returns the chat model and embedding model for the chosen provider."""
import config


def get_llm(temperature: float = 0.0):
    """Chat model used for answering, routing and grading."""
    if config.PROVIDER == "local":
        from langchain_ollama import ChatOllama

        return ChatOllama(model=config.OLLAMA_CHAT_MODEL, temperature=temperature)

    from langchain_openai import ChatOpenAI

    return ChatOpenAI(model=config.OPENAI_CHAT_MODEL, temperature=temperature)


def get_embeddings():
    """Embedding model. The SAME model must be used for indexing and querying."""
    if config.PROVIDER == "local":
        from langchain_huggingface import HuggingFaceEmbeddings

        return HuggingFaceEmbeddings(model_name=config.HF_EMBED_MODEL)

    from langchain_openai import OpenAIEmbeddings

    return OpenAIEmbeddings(model=config.OPENAI_EMBED_MODEL)
```

17.5 vectorstore.py

vectorstore.py

```
"""Single place that knows how to open the persistent Chroma vector store."""
from langchain_chroma import Chroma

import config
from models import get_embeddings


def get_vectorstore() -> Chroma:
    return Chroma(
        collection_name=config.COLLECTION,
        embedding_function=get_embeddings(),
        persist_directory=config.CHROMA_DIR,
    )


def count_chunks() -> int:
    """Number of chunks currently stored (0 means you forgot to run ingestion)."""
    return get_vectorstore()._collection.count()
```

17.6 ingest.py

ingest.py

```
"""Indexing pipeline: load files -> split into chunks -> embed -> store in Chroma."""
import shutil
import sys
from pathlib import Path
from typing import List

from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader, TextLoader
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter

import config
from models import get_embeddings


def load_documents(data_dir: str) -> List[Document]:
    """Load every .pdf / .txt / .md file found under data_dir."""
    docs: List[Document] = []
    for path in sorted(Path(data_dir).rglob("*")):
        suffix = path.suffix.lower()
        if suffix == ".pdf":
            docs.extend(PyPDFLoader(str(path)).load())  # one Document per page
        elif suffix in {".txt", ".md"}:
            docs.extend(TextLoader(str(path), encoding="utf-8").load())
    return docs


def split_documents(docs: List[Document]) -> List[Document]:
    """Cut documents into overlapping chunks, keeping metadata (source, page)."""
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=config.CHUNK_SIZE,
        chunk_overlap=config.CHUNK_OVERLAP,
        separators=["\n\n", "\n", ". ", " ", ""],
        add_start_index=True,
    )
    return splitter.split_documents(docs)


def build_index(rebuild: bool = True) -> int:
    """Create (or recreate) the vector store. Returns the number of chunks indexed."""
    if rebuild and Path(config.CHROMA_DIR).exists():
        shutil.rmtree(config.CHROMA_DIR)

    docs = load_documents(config.DATA_DIR)
    if not docs:
        sys.exit(f"No .pdf/.txt/.md files found in '{config.DATA_DIR}/'. Add some first.")

    chunks = split_documents(docs)
    print(f"Loaded {len(docs)} document(s) -> {len(chunks)} chunk(s). Embedding...")

    Chroma.from_documents(
        documents=chunks,
        embedding=get_embeddings(),
        collection_name=config.COLLECTION,
        persist_directory=config.CHROMA_DIR,
    )
    print(f"Index saved in '{config.CHROMA_DIR}/'.")
    return len(chunks)


if __name__ == "__main__":
    build_index()
```

17.7 create_sample_data.py

create_sample_data.py

```
"""Writes two small, fictional documents into data/ so you can test the projects."""
from pathlib import Path

HANDBOOK = """NovaTech Solutions - Employee Handbook (2026 Edition)

1. Working Hours
Standard working time is 40 hours per week. Core hours are 10:00 to 16:00 local time,
during which employees are expected to be available for meetings.

2. Remote Work
Employees may work from home up to 3 days per week. Fully remote arrangements require
written approval from a Vice President. Remote workers must use the company VPN.

3. Leave Policy
Every employee receives 24 days of paid annual leave per calendar year, plus 10 public
holidays. Up to 5 unused leave days may be carried over into the next year.
Sick leave is 12 days per year; a medical certificate is required after 3 consecutive days.
Parental leave is 26 weeks for the primary caregiver and 8 weeks for the secondary caregiver.

4. Travel and Expenses
Meals during business travel are reimbursed up to USD 60 per day. Flights shorter than
6 hours must be booked in economy class. Expense claims must be submitted within 30 days
through the NovaPay portal, attaching all receipts.

5. Learning Budget
Each employee has a learning budget of USD 1,200 per year for courses, books and
conferences. Unused budget does not roll over.

6. Information Security
All laptops use full-disk encryption and two-factor authentication is mandatory.
Any suspected security incident must be reported within 2 hours to the security team
at security@novatech.example.
"""

FAQ = """# NovaDesk Product FAQ

NovaDesk is NovaTech's customer help-desk software.

## Pricing and plans
- Starter: USD 12 per user per month.
- Pro: USD 29 per user per month.
- Enterprise: custom pricing, contact sales.
Every plan includes a 14-day free trial. Annual plans have a 30-day money-back guarantee.

## Support
Standard support is available Monday to Friday, 08:00 to 20:00 UTC.
Enterprise customers receive 24/7 support with a dedicated account manager.

## Reliability
Pro and Enterprise plans include a 99.9% uptime SLA. Starter has no SLA.

## Data retention
Tickets are stored for 24 months on Starter, 60 months on Pro, and a custom period on Enterprise.

## API and integrations
The API rate limit is 100 requests per minute on Starter and 1,000 requests per minute on Pro.
NovaDesk integrates with Slack, Jira and Salesforce.
"""


def main() -> None:
    data = Path("data")
    data.mkdir(exist_ok=True)
    (data / "novatech_handbook.txt").write_text(HANDBOOK, encoding="utf-8")
    (data / "novadesk_faq.md").write_text(FAQ, encoding="utf-8")
    print("Sample files written to data/")


if __name__ == "__main__":
    main()
```

17.8 state.py - the shared state

Only `messages` has a reducer (`add_messages`), because chat history must accumulate. Every other key is simply overwritten by the node that produces it.

state.py

```
"""The shared state that flows through every node of the graph."""
from typing import Annotated, List

from langchain_core.documents import Document
from langgraph.graph.message import add_messages
from typing_extensions import TypedDict


class GraphState(TypedDict):
    # Chat history. `add_messages` is a REDUCER: new messages are appended, not overwritten.
    messages: Annotated[list, add_messages]

    question: str          # what the user typed in this turn
    query: str             # standalone / rewritten query used for retrieval
    route: str             # "vectorstore" or "general"
    documents: List[Document]  # retrieved (later: graded) chunks
    generation: str        # the answer text
    grounded: bool         # did the hallucination check pass?
    retries: int           # how many rewrite loops were used in this turn
```

17.9 schemas.py - structured decisions

`Literal[...]` fields make it impossible for the router or graders to return anything but the allowed values.

schemas.py

```
"""Pydantic schemas used with `llm.with_structured_output(...)` for routing and grading."""
from typing import Literal

from pydantic import BaseModel, Field


class RouteQuery(BaseModel):
    """Decide where a user question should be answered from."""

    datasource: Literal["vectorstore", "general"] = Field(
        description="'vectorstore' for questions about the indexed documents, "
        "'general' for greetings, chit-chat or unrelated general knowledge."
    )


class GradeDocument(BaseModel):
    """Relevance check for one retrieved chunk."""

    binary_score: Literal["yes", "no"] = Field(
        description="'yes' if the document helps answer the question, otherwise 'no'."
    )


class GradeGeneration(BaseModel):
    """Hallucination check for a generated answer."""

    binary_score: Literal["yes", "no"] = Field(
        description="'yes' if the answer is fully supported by the documents "
        "(or honestly says it does not know), otherwise 'no'."
    )
```

17.10 utils.py

utils.py

```
"""Small helpers shared by the nodes."""
from pathlib import Path
from typing import List

from langchain_core.documents import Document


def source_label(doc: Document) -> str:
    name = Path(doc.metadata.get("source", "unknown")).name
    page = doc.metadata.get("page")
    return f"{name}, page {page + 1}" if page is not None else name


def format_docs(docs: List[Document]) -> str:
    """Numbered context block, e.g. '[1] (handbook.txt)\\n...text...'."""
    return "\n\n".join(
        f"[{i}] ({source_label(d)})\n{d.page_content}" for i, d in enumerate(docs, start=1)
    )
```

17.11 nodes.py - all the logic

nodes.py

```
"""All graph nodes (functions that return partial state updates) and routing functions."""
from functools import lru_cache

from langchain_core.messages import AIMessage
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

import config
from models import get_llm
from schemas import GradeDocument, GradeGeneration, RouteQuery
from state import GraphState
from utils import format_docs
from vectorstore import get_vectorstore

# ---------------------------------------------------------------------------
# Lazy singletons (created on first use, then reused)
# ---------------------------------------------------------------------------


@lru_cache(maxsize=1)
def _llm():
    return get_llm()


@lru_cache(maxsize=1)
def _retriever():
    return get_vectorstore().as_retriever(search_kwargs={"k": config.TOP_K})


def _history(state: GraphState):
    """Previous chat messages (everything except the newest user message)."""
    return state["messages"][:-1][-config.HISTORY_MESSAGES:]


# ---------------------------------------------------------------------------
# Prompts
# ---------------------------------------------------------------------------

CONDENSE_PROMPT = ChatPromptTemplate.from_messages(
    [
        ("system",
         "Given the chat history and the latest user question, rewrite the latest question "
         "as a standalone question that can be understood without the history. "
         "Do NOT answer it. If it is already standalone, return it unchanged. "
         "Return only the question."),
        MessagesPlaceholder("history"),
        ("human", "{question}"),
    ]
)

ROUTE_PROMPT = ChatPromptTemplate.from_messages(
    [
        ("system",
         "You are a router. The vector store contains {kb}. "
         "Choose 'vectorstore' for any question that could be answered by those documents. "
         "Choose 'general' for greetings, thanks, chit-chat, or general knowledge unrelated "
         "to those documents."),
        ("human", "{query}"),
    ]
)

GRADE_DOC_PROMPT = ChatPromptTemplate.from_messages(
    [
        ("system",
         "You grade whether a retrieved document is relevant to a user question. "
         "It is relevant if it contains facts, keywords or meaning that help answer the "
         "question. Answer 'yes' or 'no'."),
        ("human", "Document:\n{document}\n\nQuestion: {query}"),
    ]
)

REWRITE_PROMPT = ChatPromptTemplate.from_messages(
    [
        ("system",
         "You improve search queries for a vector database. Rewrite the query so it is more "
         "likely to match relevant passages: use precise keywords and synonyms, keep the "
         "original intent. Return only the rewritten query."),
```

```
        ("human", "Original query: {query}"),
    ]
)

GENERATE_PROMPT = ChatPromptTemplate.from_messages(
    [
        ("system",
         "You are a precise assistant that answers using ONLY the context provided.\n"
         "Rules:\n"
         "1. If the context does not contain the answer, say you don't know based on the "
         "documents.\n"
         "2. Be concise and factual. Never invent details.\n"
         "3. Cite sources by their numbers, for example [1] or [2]."),
        MessagesPlaceholder("history"),
        ("human", "Context:\n{context}\n\nQuestion: {question}"),
    ]
)

GRADE_GEN_PROMPT = ChatPromptTemplate.from_messages(
    [
        ("system",
         "You check whether an ANSWER is grounded in the FACTS. Reply 'yes' if every claim "
         "in the answer is supported by the facts (or the answer honestly says it does not "
         "know). Reply 'no' if the answer contains unsupported claims."),
        ("human", "FACTS:\n{documents}\n\nANSWER:\n{generation}"),
    ]
)

DIRECT_PROMPT = ChatPromptTemplate.from_messages(
    [
        ("system",
         "You are a friendly assistant. Answer briefly. You do not have access to company "
         "documents in this mode, so do not state company facts; suggest asking a "
         "document-related question instead."),
        MessagesPlaceholder("history"),
        ("human", "{question}"),
    ]
)

# ---------------------------------------------------------------------------
# Nodes
# ---------------------------------------------------------------------------


def condense_question(state: GraphState) -> dict:
    """Turn a follow-up like 'and for Pro?' into a standalone question."""
    history = _history(state)
    if not history:
        return {"query": state["question"]}
    chain = CONDENSE_PROMPT | _llm() | StrOutputParser()
    standalone = chain.invoke({"history": history, "question": state["question"]})
    return {"query": standalone.strip()}


def route_question(state: GraphState) -> dict:
    """LLM decides: search the knowledge base or answer directly."""
    router = ROUTE_PROMPT | _llm().with_structured_output(RouteQuery)
    decision = router.invoke({"kb": config.KB_DESCRIPTION, "query": state["query"]})
    return {"route": decision.datasource}


def retrieve(state: GraphState) -> dict:
    return {"documents": _retriever().invoke(state["query"])}


def grade_documents(state: GraphState) -> dict:
    """Keep only the chunks the grader considers relevant (runs in parallel via batch)."""
    grader = GRADE_DOC_PROMPT | _llm().with_structured_output(GradeDocument)
    docs = state["documents"]
    inputs = [{"document": d.page_content, "query": state["query"]} for d in docs]
    verdicts = grader.batch(inputs) if inputs else []
    relevant = [d for d, v in zip(docs, verdicts) if v.binary_score == "yes"]
    return {"documents": relevant}


def rewrite_query(state: GraphState) -> dict:
    better = (REWRITE_PROMPT | _llm() | StrOutputParser()).invoke({"query": state["query"]})
    return {"query": better.strip(), "retries": state["retries"] + 1}
```

```
def generate(state: GraphState) -> dict:
    chain = GENERATE_PROMPT | _llm() | StrOutputParser()
    answer = chain.invoke(
        {
            "history": _history(state),
            "context": format_docs(state["documents"]),
            "question": state["question"],
        }
    )
    return {"generation": answer}


def grade_generation(state: GraphState) -> dict:
    """Hallucination check: is the answer supported by the retrieved documents?"""
    checker = GRADE_GEN_PROMPT | _llm().with_structured_output(GradeGeneration)
    verdict = checker.invoke(
        {"documents": format_docs(state["documents"]), "generation": state["generation"]}
    )
    return {"grounded": verdict.binary_score == "yes"}


def direct_answer(state: GraphState) -> dict:
    chain = DIRECT_PROMPT | _llm() | StrOutputParser()
    answer = chain.invoke({"history": _history(state), "question": state["question"]})
    return {"generation": answer}


def fallback(state: GraphState) -> dict:
    return {
        "generation": (
            "I couldn't find relevant information in the knowledge base for that question. "
            "Try rephrasing it or ask about topics covered by the documents."
        )
    }


def finalize(state: GraphState) -> dict:
    """Last node of every path: store the answer in the chat history exactly once."""
    text = state["generation"]
    if not state.get("grounded", True):
        text += "\n\n(Note: this answer could not be fully verified against the documents.)"
    return {"generation": text, "messages": [AIMessage(content=text)]}


# ---------------------------------------------------------------------------
# Routing functions used by conditional edges (they only READ the state)
# ---------------------------------------------------------------------------


def decide_route(state: GraphState) -> str:
    return state["route"]  # "vectorstore" | "general"


def decide_after_grading(state: GraphState) -> str:
    if state["documents"]:
        return "generate"
    if state["retries"] < config.MAX_RETRIES:
        return "rewrite"
    return "fallback"


def decide_after_generation(state: GraphState) -> str:
    if state["grounded"] or state["retries"] >= config.MAX_RETRIES:
        return "finalize"
    return "rewrite"
```

17.12 graph.py - wiring

graph.py

```
"""Assembles the nodes and edges into a compiled LangGraph application."""
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph

import nodes
from state import GraphState


def build_graph():
    g = StateGraph(GraphState)

    # --- nodes -------------------------------------------------------------
    g.add_node("condense_question", nodes.condense_question)
    g.add_node("route_question", nodes.route_question)
    g.add_node("retrieve", nodes.retrieve)
    g.add_node("grade_documents", nodes.grade_documents)
    g.add_node("rewrite_query", nodes.rewrite_query)
    g.add_node("generate", nodes.generate)
    g.add_node("grade_generation", nodes.grade_generation)
    g.add_node("direct_answer", nodes.direct_answer)
    g.add_node("fallback", nodes.fallback)
    g.add_node("finalize", nodes.finalize)

    # --- edges -------------------------------------------------------------
    g.add_edge(START, "condense_question")
    g.add_edge("condense_question", "route_question")

    g.add_conditional_edges(
        "route_question",
        nodes.decide_route,
        {"vectorstore": "retrieve", "general": "direct_answer"},
    )

    g.add_edge("retrieve", "grade_documents")
    g.add_conditional_edges(
        "grade_documents",
        nodes.decide_after_grading,
        {"generate": "generate", "rewrite": "rewrite_query", "fallback": "fallback"},
    )
    g.add_edge("rewrite_query", "retrieve")  # the retry loop

    g.add_edge("generate", "grade_generation")
    g.add_conditional_edges(
        "grade_generation",
        nodes.decide_after_generation,
        {"finalize": "finalize", "rewrite": "rewrite_query"},
    )

    g.add_edge("direct_answer", "finalize")
    g.add_edge("fallback", "finalize")
    g.add_edge("finalize", END)

    # MemorySaver keeps one checkpoint history per thread_id (conversation memory).
    return g.compile(checkpointer=MemorySaver())
```

17.13 main.py - CLI with live trace

main.py

```
"""Command-line interface for Project 2 (agentic RAG with LangGraph).

Usage:
    python create_sample_data.py
    python main.py ingest
    python main.py ask "What is the Pro plan price?"
    python main.py chat          # multi-turn chat with memory + live node trace
    python main.py graph         # print the graph as a Mermaid diagram
"""
import argparse
import uuid

from langchain_core.messages import HumanMessage

from graph import build_graph
from ingest import build_index
from utils import source_label
from vectorstore import count_chunks


def run_turn(graph, question: str, thread_id: str, verbose: bool = True) -> dict:
    """Run one user turn through the graph and return the merged final updates."""
    run_config = {"configurable": {"thread_id": thread_id}, "recursion_limit": 40}
    inputs = {
        "messages": [HumanMessage(content=question)],
        "question": question,
        "documents": [],
        "generation": "",
        "grounded": True,
        "retries": 0,
    }
    final: dict = {}
    # stream_mode="updates" yields {node_name: partial_state_update} after each node.
    for update in graph.stream(inputs, run_config, stream_mode="updates"):
        for node_name, delta in update.items():
            if verbose:
                print(f"   -> {node_name}")
            if delta:
                final.update(delta)
    return final


def show(final: dict) -> None:
    print(f"\nAssistant: {final.get('generation', '(no answer)')}")
    docs = final.get("documents") or []
    if docs:
        print("Sources  :", "; ".join(dict.fromkeys(source_label(d) for d in docs)))


def require_index() -> None:
    if count_chunks() == 0:
        raise SystemExit("Vector store is empty. Run:  python main.py ingest")


def main() -> None:
    parser = argparse.ArgumentParser(description="Agentic RAG with LangGraph")
    sub = parser.add_subparsers(dest="command", required=True)
    sub.add_parser("ingest")
    ask_cmd = sub.add_parser("ask")
    ask_cmd.add_argument("question")
    sub.add_parser("chat")
    sub.add_parser("graph")
    args = parser.parse_args()

    if args.command == "ingest":
        build_index()
        return

    if args.command == "graph":
        print(build_graph().get_graph().draw_mermaid())
        return

    require_index()
    graph = build_graph()

    if args.command == "ask":
        show(run_turn(graph, args.question, thread_id=str(uuid.uuid4())))
```

```
        return

    thread_id = str(uuid.uuid4())  # one conversation = one thread
    print("Agentic RAG chat. Type 'exit' to quit, 'new' to start a fresh conversation.")
    while True:
        try:
            question = input("\nYou: ").strip()
        except (EOFError, KeyboardInterrupt):
            print()
            break
        if question.lower() in {"exit", "quit"}:
            break
        if question.lower() == "new":
            thread_id = str(uuid.uuid4())
            print("(new conversation started)")
            continue
        if question:
            show(run_turn(graph, question, thread_id))


if __name__ == "__main__":
    main()
```

18. Project 2 - run, trace and understand

18.1 Running

terminal

```
python create_sample_data.py
python main.py ingest
python main.py graph                        # prints the Mermaid diagram of the graph
python main.py ask "What is the Pro plan price?"
python main.py chat
```

Illustrative chat showing the node trace for each path (names appear as each node finishes):

example session

```
You: How many days of annual leave do I get?
   -> condense_question
   -> route_question
   -> retrieve
   -> grade_documents
   -> generate
   -> grade_generation
   -> finalize
Assistant: You get 24 days of paid annual leave per calendar year, plus 10 public holidays. [1]
Sources  : novatech_handbook.txt

You: Can I carry some over?                      <-- follow-up, understood via chat history
   -> condense_question                          (query is now a standalone question)
   -> route_question
   ...
Assistant: Yes, up to 5 unused leave days may be carried over into the next year. [1]

You: Hello!
   -> condense_question
   -> route_question                             (route = general)
   -> direct_answer
   -> finalize
Assistant: Hi! Ask me anything about the NovaTech handbook or the NovaDesk product.

You: What is the salary of the CEO?
   -> condense_question
   -> route_question
   -> retrieve
   -> grade_documents                            (no chunk is relevant)
   -> rewrite_query                              (retry 1)
   -> retrieve
   -> grade_documents
   -> rewrite_query                              (retry 2)
   -> retrieve
   -> grade_documents
   -> fallback                                   (retries exhausted)
   -> finalize
Assistant: I couldn't find relevant information in the knowledge base for that question. ...
```

> **NOTE**
> Your traces may differ slightly: LLM routing and grading are probabilistic. The *shape* - bounded retries ending in `fallback` - is guaranteed by the graph.

18.2 A test plan that covers every path

Input	Path exercised	Expected behaviour
How many leave days do employees get?	route -> retrieve -> grade -> generate -> grade_generation -> finalize	24 days with citation
And what about sick leave?	condense_question rewrites the follow-up	12 days; medical certificate after 3 days

Input	Path exercised	Expected behaviour
Hi there!	route -> direct_answer	Friendly reply, no retrieval
What is the CEO's salary?	grade -> rewrite loop -> fallback	Polite 'not found' after 2 retries
Which plan has the 99.9% SLA and how much is it?	multi-fact retrieval	Pro (USD 29) and Enterprise (custom)
Thanks!	route -> direct_answer	Short acknowledgement

18.3 How the pieces fit - things worth understanding deeply

1. **State is the contract.** Every node's input and output is a slice of `GraphState`. If you add a feature, first ask which new state key it needs.
2. **Each turn re-initialises the per-turn keys** (`documents`, `generation`, `grounded`, `retries`) in `run_turn`. Only `messages` persists across turns because the checkpointer keeps the thread's state and the reducer appends.
3. **Conditional edges only read state.** `decide_after_grading` looks at `documents` and `retries`; it never calls an LLM. The LLM work happens in nodes, so every decision is visible in the trace.
4. **The retry loop is bounded twice**: `retries < MAX_RETRIES` in the routing functions and `recursion_limit` in the run config. Remove the first and the second still protects you with an error rather than an infinite bill.
5. `finalize` **is the single exit** for every path. It writes the answer into `messages` exactly once, so retries never create duplicate AI messages in the history.
6. `grade_documents` **uses** `batch` so grading the k chunks runs concurrently - latency is roughly one grader call, not k calls.
7. `thread_id` **= conversation.** A new UUID gives a fresh memory; reuse it to continue. In a web app use the user's session or conversation id.

18.4 Cost and latency

A normal grounded answer costs about 3 + k small LLM calls (condense if history, route, k graders, generate, check) versus 1 call in Project 1. That is the price of reliability. Reduce it by: grading with a small model (`get_llm()` for grading only), lowering `TOP_K`, skipping `grade_generation` for low-risk questions, caching routes for repeated questions, or replacing per-chunk grading with a cross-encoder re-ranker.

18.5 Troubleshooting

Problem	Fix
`GraphRecursionError`	A loop did not terminate: check `retries` is incremented in `rewrite_query` and `MAX_RETRIES` is set.
Every question goes to `general`	Edit `KB_DESCRIPTION` in `config.py` to describe your documents.
Every chunk graded 'no'	Grader model too small or chunks too short; try a stronger model for grading, larger chunks, or temporarily skip grading.
`ValueError: ... is already being used as a state key`	A node name equals a state key. Rename the node (e.g. `generate` vs `generation`).
Structured-output / schema error with a local model	Use a recent Ollama and a model that supports JSON/tool output (e.g. llama3.1); otherwise use OpenAI for grading.

Problem	Fix
Memory not remembered	You used a new `thread_id` each time, or restarted the process (`MemorySaver` is in-memory).
`KeyError: 'retries'`	The input to `graph.stream` must initialise `retries` (see `run_turn`).

18.6 Extension ideas

- **Web-search fallback** (Corrective RAG): replace `fallback` with a node that calls a search tool, then generates.
- **Persistent memory:** swap `MemorySaver` for `SqliteSaver` or `PostgresSaver`.
- **Human approval** before answering sensitive questions using `interrupt()`.
- **Re-ranker** after `retrieve` (cross-encoder) and metadata filters from the user's permissions.
- **Streaming UI:** `stream_mode='messages'` with Streamlit/Gradio/FastAPI + Server-Sent Events.
- **Evaluation:** run a 50-question test set through `run_turn` and score faithfulness and correctness with RAGAS or LangSmith.
- **Multi-agent:** make the retrieval part a sub-graph used by a supervisor graph that also has SQL and calculator tools.

PART 5

Reference and Practice

Cheat sheets, interview questions, exercises, glossary and resources

19. Cheat sheets

19.1 RAG pipeline in 12 lines (LangChain)

minimal_rag.py

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

docs = PyPDFLoader("file.pdf").load()
chunks = RecursiveCharacterTextSplitter(chunk_size=600, chunk_overlap=100).split_documents(docs)
vs = Chroma.from_documents(chunks, OpenAIEmbeddings(model="text-embedding-3-small"))
retriever = vs.as_retriever(search_kwargs={"k": 4})
prompt = ChatPromptTemplate.from_template("Answer ONLY from context.\n{context}\nQ: {question}")
ask = lambda q: (prompt | ChatOpenAI(model="gpt-4o-mini") | StrOutputParser()).invoke(
    {"context": "\n\n".join(d.page_content for d in retriever.invoke(q)), "question": q})
print(ask("What is in the file?"))
```

19.2 LangGraph API at a glance

Task	Code
Define state	`class State(TypedDict): x: int`
Reducer (append)	`Annotated[list, operator.add]` or `Annotated[list, add_messages]`
Create builder	`b = StateGraph(State)`
Add node	`b.add_node('name', fn)`
Fixed edge	`b.add_edge('a', 'b')` (use `START`, `END` for entry/exit)
Conditional edge	`b.add_conditional_edges('a', router_fn, {'x': 'node_x', 'y': END})`
Compile	`g = b.compile(checkpointer=MemorySaver())`
Run	`g.invoke(input, {'configurable': {'thread_id': '1'}})`
Stream	`for u in g.stream(input, cfg, stream_mode='updates'): ...`
Inspect state	`g.get_state(cfg).values`
Pause for human	`value = interrupt(payload)`; resume with `g.invoke(Command(resume=v), cfg)`
Tools	`llm.bind_tools(tools)`, `ToolNode(tools)`, `tools_condition`
Loop guard	`cfg = {..., 'recursion_limit': 25}`
Draw	`print(g.get_graph().draw_mermaid())`

19.3 Parameter cheat sheet

Knob	Start with	Tune when
chunk_size / overlap	600-1000 chars / 10-20%	Answers are cut off (larger) or noisy (smaller)
top-k	4	Recall low (up) or cost/noise high (down)
temperature	0	Never raise for factual QA
MAX_RETRIES	2	Latency matters (1) or hard corpus (3)
Embedding model	text-embedding-3-small	Multilingual / domain-specific text

Knob	Start with	Tune when
Search type	similarity	Duplicates (mmr), keywords/IDs (hybrid)

20. Interview questions and model answers

Q1. What is RAG and why use it instead of fine-tuning?

RAG retrieves relevant documents at query time and gives them to the LLM as context. It keeps knowledge fresh and updatable, reduces hallucination, enables citations and per-user access control, and is cheaper than retraining. Fine-tuning is better for style/behaviour, not for injecting changing facts.

Q2. Walk me through a RAG pipeline.

Offline: load, split into chunks, embed, store in a vector DB. Online: embed the question, retrieve top-k, build a prompt with the chunks, generate with a low-temperature LLM, return answer plus sources.

Q3. How do you choose chunk size and overlap?

Start around 500-1000 tokens-equivalent with 10-20% overlap, then measure retrieval hit rate/MRR on a labelled question set and adjust. Structure-aware or parent-child chunking helps for structured documents.

Q4. What is an embedding and which similarity metric do you use?

A dense vector representing meaning; similar texts are close. Cosine similarity (or dot product on normalised vectors) is standard. The same model must embed documents and queries.

Q5. What is the difference between dense, sparse and hybrid retrieval?

Dense = embeddings (semantic). Sparse = BM25/TF-IDF keywords (exact terms, IDs). Hybrid fuses both, often with Reciprocal Rank Fusion, and is more robust.

Q6. What does re-ranking do?

A cross-encoder scores query-document pairs jointly, which is more accurate than vector distance but slower, so you re-rank only the top 20-50 candidates and keep the best few.

Q7. How do you reduce hallucinations in RAG?

Grounding prompt with a refusal rule, low temperature, better retrieval (hybrid, re-rank, grading), score thresholds, citations, and a groundedness check on the answer (Self-RAG).

Q8. How do you evaluate a RAG system?

Separate retrieval (hit rate, recall@k, MRR, context precision) from generation (faithfulness, answer relevance, correctness). Use a golden dataset and tools like RAGAS, TruLens or LangSmith; re-run on every change.

Q9. How do you handle follow-up questions in a chatbot?

Condense the latest message plus chat history into a standalone query before retrieval (Project 2's `condense_question` node).

Q10. What is LangGraph and how is it different from a LangChain chain?

LangGraph builds stateful graphs with nodes, edges, conditional routing and loops, plus checkpointing, streaming and human-in-the-loop. Chains are linear pipelines.

Q11. What is a reducer in LangGraph?

A function that defines how a node's update is merged into a state key, e.g. `add_messages` appends to the message list; keys without a reducer are overwritten.

Q12. How do conditional edges work?

A routing function receives the state and returns a label; a mapping converts the label into the next node name.

Q13. How does LangGraph implement memory?

A checkpointer saves state after each step per `thread_id`; reusing the id restores the conversation. Use SQLite/Postgres savers for durability.

Q14. What is Corrective RAG / Self-RAG / Adaptive RAG?

CRAG grades retrieved docs and corrects retrieval (rewrite, web search) before answering. Self-RAG checks the generated answer for support and usefulness. Adaptive RAG routes questions by complexity or type.

Q15. How do you prevent infinite loops in agentic graphs?

Counter in state checked by routing functions, `recursion_limit` in the config, and a terminal fallback path.

Q16. What security issues exist in RAG?

Prompt injection via retrieved text, data leakage across users (enforce filters at retrieval), PII in the index, and stale or deleted documents remaining indexed.

21. Practice exercises

21.1 Beginner

1. Index three PDFs of your own and write 10 questions with expected answers; measure hit rate with `eval_retrieval.py`.
2. Print the chunks for one question (`result['docs']`) and decide by eye which chunk size would have worked better.
3. Add the `--k` command-line option to `main.py ask`.
4. Draw the Project 1 pipeline on paper from memory, then compare with Chapter 2.

21.2 Intermediate

1. Switch Project 1 to MMR retrieval and compare hit rate and answer quality.
2. Add a cross-encoder re-ranker (`sentence-transformers CrossEncoder`) between retrieval and prompt.
3. Add hybrid search: BM25 (`rank_bm25`) + vector results fused with the RRF function from Section 3.5.
4. In Project 2, change `MAX_RETRIES` to 0 and 3 and observe traces for an unanswerable question.
5. Replace `MemorySaver` with `SqliteSaver` and prove memory survives a restart.

21.3 Advanced

1. Add a `web_search` node as the Corrective-RAG fallback and merge its results into `documents`.
2. Add an `interrupt()` before `generate` for questions tagged confidential and build a tiny approval UI.
3. Build an evaluation script that runs 50 questions through `run_turn` and reports faithfulness with an LLM judge.
4. Turn the retrieval part (`retrieve`, `grade_documents`, `rewrite_query`) into a sub-graph and call it from a supervisor graph.
5. Deploy the graph behind FastAPI with streaming responses (`stream_mode='messages'`).

22. Glossary

Term	Meaning
Agentic RAG	RAG where an LLM-driven graph decides whether/how to retrieve, checks results and retries.
ANN / HNSW	Approximate nearest-neighbour search; HNSW is a popular graph-based index.
BM25	Classic keyword ranking function (sparse retrieval).
Checkpointer	LangGraph component that saves state after each step (memory/resume).
Chunk	A piece of a document that is embedded and retrieved as a unit.
Conditional edge	A graph edge whose target is chosen at run time by a routing function.
Context window	Maximum number of tokens a model can read in one request.
Cosine similarity	Similarity based on the angle between two vectors.
Cross-encoder	Model scoring a query and a document together; used for re-ranking.
Embedding	Numeric vector representing the meaning of text.
Faithfulness / groundedness	Share of answer claims supported by the retrieved context.
Grounding	Constraining an answer to supplied evidence.
HyDE	Hypothetical Document Embeddings: embed a generated draft answer to retrieve.
LCEL	LangChain Expression Language: composing runnables with the | operator.
MMR	Maximal marginal relevance: balances relevance and diversity.
Node / Edge / State	LangGraph building blocks: function, transition, shared data.
Reducer	Rule for merging a node's update into a state key.
Retriever	Component that maps a query to relevant documents.
RRF	Reciprocal Rank Fusion: merges ranked lists.
Thread (thread_id)	Identifier of one conversation's saved state in LangGraph.
Top-k	Number of chunks retrieved per query.
Vector store	Database optimised for similarity search over embeddings.

23. Where to go next

- Official docs: **python.langchain.com** (LangChain) and **langchain-ai.github.io/langgraph** (LangGraph) - including the 'Agentic RAG' and 'Adaptive RAG' tutorials.
- Original RAG paper: Lewis et al., 2020, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.
- Corrective RAG (Yan et al., 2024) and Self-RAG (Asai et al., 2023) papers for the patterns in Project 2.
- **MTEB leaderboard** (Hugging Face) for choosing embedding models; **RAGAS** and **LangSmith** for evaluation.
- Suggested capstone: a RAG assistant for your own college, company or course material, with a test set, metrics table, and a short write-up of what you changed and what improved the numbers.

FINAL ADVICE

The three habits that separate demos from products: (1) a labelled evaluation set, (2) looking at the retrieved chunks whenever an answer is wrong, (3) bounded, observable control flow. You now have working code for all three.